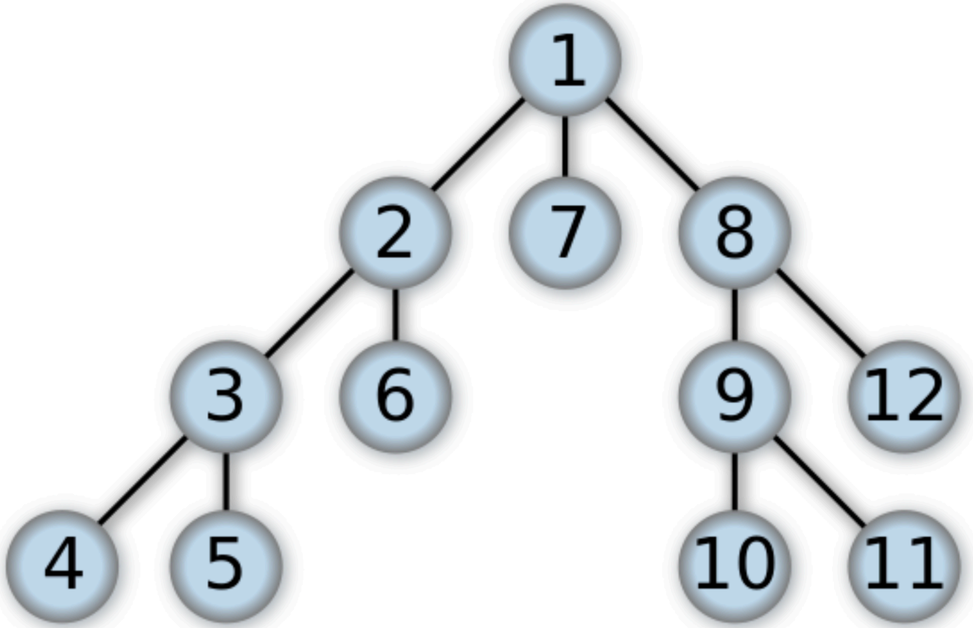
Graphs - Extra Notes

Rodrigo Canaan
CSC 349 Fall 2023

Depth-First Search (DFS)

Depth-first search



- Search deeply first
- Fringe is a LIFO stack

Depth-First Search

Consider the explore algorithm with pre- and postorderings.

---

explore(G, v)

Input: A graph G and a vertex v.
Output: Vertices labeled "discovered" are vertices reachable from v.

```
1: discovered(v) =true.
2: previsit(v)
3: for all neighbors of v, u do
4:     if discovered(u) =false then
5:         explore(G, u)
6: postvisit(v)
```

Running Time for DFS

DFS(G)

Input: A graph $G = (V, E)$.
Output: A *forest* of connected components of G.

```
1: for all v ∈ V do
2:     discovered(v) =false
3: for all v ∈ V do
4:     if discovered(v) =false then
5:         explore(G, v)
```

- Step 1 takes $|V|$ time.
- We call explore(G, v) $|V|$ times (once for each vertex).
- In explore, we examine all neighbors of a vertex, so we examine each edge (twice), $2|E|$.

The time complexity is $2|V| + 2|E| = O(|V| + |E|) = O(n + m)$

Depth-First Search in Directed Graphs

The algorithm runs with one small change to explore:

---

explore(G, v)

Input: A **directed** graph G and a vertex v.
Output: Vertices labeled "discovered" are vertices reachable from v.

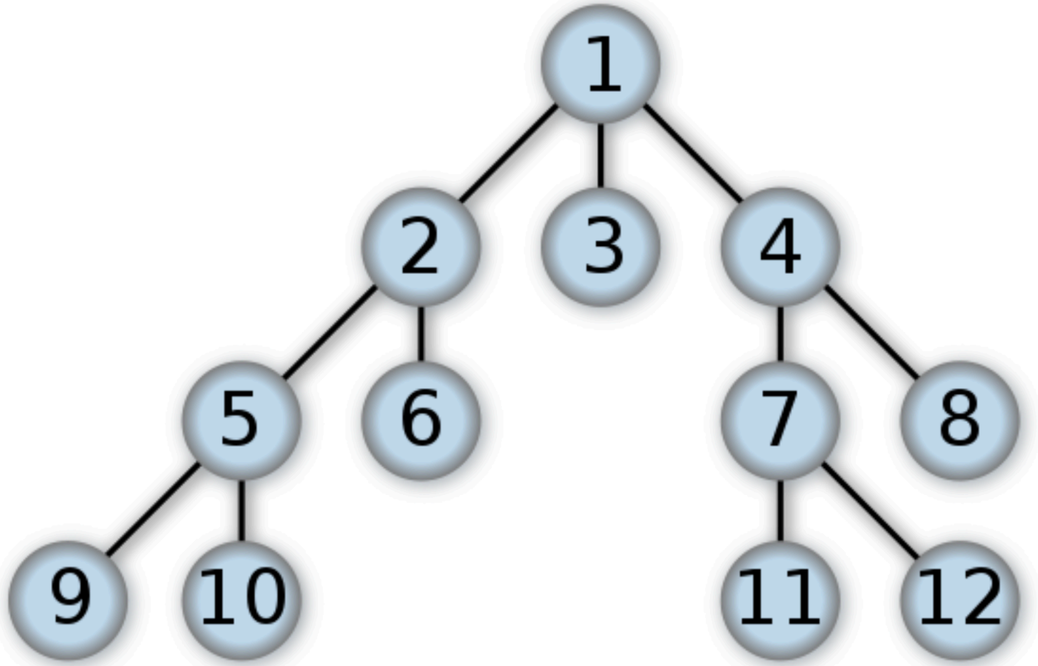
```
1: discovered(v) =true.
2: for all outgoing neighbors of v, u do
3:     if discovered(u) =false then
4:         explore(G, u)
```

Some equivalences to take note (DFS)

- If $G = \{V, E\}$ is a simple (undirected) graph and u, v are vertices in V, the following statements are all equivalent:
  - There is a path from u to v
  - u and v are connected
  - u and v are in the same connected component
  - $DFS - Visit(G, u)$ discovers v
  - After DFS(G), either u or v will belong to the other's subtree

Breadth-First Search (BFS)

Breadth-first search



- Search widely first
- Fringe is a FIFO queue

Running Time for BFS

BFS(G, A)

Input: A graph $G = (V, E)$ and a vertex A.
Output: For all vertices, X, reachable from A, $dist(X)$ is set to be the distance from A to X.

```
1:  for all X ∈ V do
2:      dist(X) = ∞
3:  dist(A) = 0
4:  Q = [A] (a queue containing A)
5:  while Q is not empty do
6:      X = dequeue(Q)
7:      for all edges (X, Y) ∈ E do
8:          if dist(Y) = ∞ then
9:              enqueue(Q, Y)
10:             dist(Y) = dist(X) + 1
```

- Step 1 takes $|V|$ time.
- Each vertex gets placed in the queue at most once. $|V|$ time.
- In Step 7, we examine all neighbors of a vertex, so, at most, we examine each edge (twice), $2|E|$.

The time complexity is $2|V| + 2|E| = O(|V| + |E|) = O(n + m)$

	DFS	BFS
Time Complexity (worst case)		
Optimal (i.e. shortest) path?		
Space Complexity		
Data Structure		

	DFS	BFS
Time Complexity (worst case)	O(n+m)	O(n+m)
Optimal (i.e. shortest) path?		
Space Complexity		
Data Structure		

	DFS	BFS
Time Complexity (worst case)	O(n+m)	O(n+m)
Optimal (i.e. shortest) path?	No	Yes
Space Complexity		
Data Structure		

Tree Search and Space Complexity

- So far we've assumed your graph fits in memory and is given to you as an adjacency list of matrix.
- In this case, memory usage of your algorithm is typically a minor consideration: at worst, you end up storing a copy of every vertex and/or edge in the queue/stack and visited set
- In regular graph search neither DFS nor BFS has a clear space advantage
    - They both have to store all visited vertices.

Tree Search and Space Complexity

- However if your graph is too big (e.g. crawling the web) or has to be generated on the fly (e.g. simulating move sequences in a game of chess), space can become a bottleneck

- If you can’t store all visited vertices, you an’t reliably check if a specific vertex has been visited. Therefore you can’t detect cycles. Your search space becomes like a tree (no cycles)

- We call *Tree Search* the process of searching a graph without tracking repeat nodes

Tree Search and Space Complexity

- Suppose your graph is a rooted tree
  - We call b the average *branching factor* (children per non-leaf node) of the tree
  - We call d *the* maximum *depth of the tree*
- It is typical to assume (for simplicity) that the tree is uniform: all non-leaf vertices have b children, and all branches go to depth d.

Tree Search and Space Complexity

- In a uniform tree with depth d and branching factor b:
  - DFS only keeps one branch of the tree (and possibly neighbors along the way) in the stack.
    - This costs $O(b * d)$: the branch has depth d, each level has b neighbors
  - If BFS has just expanded all vertices at depth $d - 1$, however, it will have stored in the queue all nodes at depth d.
    - This costs $O(b^d)$: every subsequent level has b times as many vertices as the last

	DFS	BFS
Time Complexity (worst case)	O(n+m)	O(n+m)
Optimal (i.e. shortest) path?	No	Yes
Space Complexity (tree search)	O(b*d)	O(b^d)
Data Structure		

	DFS	BFS
Time Complexity (worst case)	O(n+m)	O(n+m)
Optimal (i.e. shortest) path?	No	Yes
Space Complexity (tree search)	O(d*b)	O(b^d)
Data Structure	Stack (LIFO)	Queue (FIFO)

Extra notes

- I assumed DFS has to store the neighbors at every level.
  - For some problems, if you can list children in a consistent order, it is enough to store an index at each node
  - DFS space complexity becomes $O(d)$ rather than $O(b * d)$
- See backtracking search in ch.3 of Russel & Norvig's AI - A Novel Approach

Extra notes

- Running DFS with a hard depth cutoff is called Depth-Limited DFS
- We can also repeatedly call Depth-Limited DFS with higher and higher cutoffs. This is called Iterative Deepening DFS (IDDFS)
  - This preserves the memory advantages of BFS and the property that DFS finds the optimal path to any vertex (in the original graph)
  - We must discard the results of searches at lower cutoff (to keep memory low), but this turns out to represent a modest overhead
  - We may still set a hard cutoff dictated by our budget.
- See Depth-Limited Search and Iterative Deepening in ch.3 of Russel & Norvig's AI - A Novel Approach